

Finishing Labs 1 - 4: a click-by-click guide

Written for Anas – follow it from top to bottom, one week at a time.

How this works

All the **code** for Labs 1–4 is already written and tested, and each week's **worklog** is already written for you. What's left needs **your** accounts and **your** laptop: make the free API keys, run the notebooks/scripts (they talk to real AI models, so they can't run without your keys), take the screenshots, and press one command that fills the worklog with your real numbers.

The worklogs have **yellow placeholders** where your results go. You never type numbers in by hand: the notebooks save their results to a **results** folder, and `build_worklogs.py` copies them into the worklog.

What the symbols mean

You'll see...	It means...
<code>Cmd</code> + <code>S</code>	Press these keys together (hold the first, tap the second).
Kernel > Restart Kernel and Run All Cells...	Click the Kernel menu, then that item.
<code>black box</code>	A command: type it exactly (or copy-paste) into Terminal, then press <code>Enter</code> .
SCREENSHOT 3.4	Take a screenshot now and save it with the name shown.
TIP / WATCH OUT	Helpful extra info / something that commonly goes wrong.
On Windows: grey box	Only read it if you're on a Windows laptop – this guide is for a Mac.

Time you'll need (roughly)

Part	What you'll do	Time
0 · Setup	Get the new files, install the Lab 1–3 libraries, make your <code>.env</code> key file	≈ 20 min
Week 1	Ollama + Gemini key, run the Lab 1 notebook, screenshots	≈ 45 min
Week 2	Hugging Face token, run the smolagents notebook (local model is slow)	≈ 1 h
Week 3	Pinecone + Make.com (in the browser), then the Python RAG scripts	≈ 1.5 h
Week 4	Separate Python 3.12 environment, run the AutoGen app + scripts	≈ 45 min
Finish	Hand-in copies, save everything to GitHub	≈ 15 min

Your progress tracker

Tick things off as you go.

	Task	Done?
Part 0	New files downloaded (git pull)	☐
	Lab 1-3 libraries installed	☐
	.env file made (keys go in later)	☐
Week 1	Ollama installed + llama3.2:1b in Terminal (1.1)	☐
	Gemini key in .env (1.3)	☐
	Lab1 notebook run (1.2, 1.4-1.6, 1.8)	☐
	Gradio chat (1.7)	☐
	Worklog finished + hand-in copy	☐
Week 2	Hugging Face token in .env (2.1)	☐
	smolagents installed (2.2)	☐
	Lab2 notebook run (2.3-2.7)	☐
	GradioUI chat (2.8)	☐
	Worklog finished + hand-in copy	☐
Week 3	Pinecone index + key in .env (3.1)	☐
	Make: ingestion (3.2, 3.3)	☐
	Make: query (3.4-3.6)	☐
	Python: concept demo, ingest, query (3.7-3.11)	☐
	experiments.py + make_check.py (3.12)	☐
	Worklog finished + hand-in copy	☐
Week 4	Python 3.12 environment + install (4.1)	☐
	Fix screenshot (4.2)	☐
	App + 4 examples (4.3-4.6)	☐
	run_examples + groupchat_demo (4.7, 4.8)	☐
	Worklog finished + hand-in copy	☐
Finish	Everything pushed to GitHub	☐
	Handed in what my tutor asked for	☐

Part 0 · Setup

≈ 20 minutes · once for all four labs

TIP You already did the big setup for Labs 5-9 (Python 3.12 + the **.venv** environment in **Documents** > **LabChallenge**). This part only adds what's new.

0.1 Open Terminal in your LabChallenge folder

- 1 Press `Cmd` + `Space`, type **Terminal**, press `Enter`.
- 2 Type `cd` (with a space after it), then **drag your LabChallenge folder** from Finder into the Terminal window, and press `Enter`.
Or type `cd ~/Documents/LabChallenge` if that's where it is.
- 3 Switch your Python environment on:

```
TYPE THIS, then press Enter
source .venv/bin/activate
```

The line now starts with **(.venv)**. ✓ Do this every time you open a new Terminal window for these labs (except Week 4, which has its own).

On Windows: open the LabChallenge folder in File Explorer, click the address bar, type `cmd` and press `Enter`; activate with `.venv\Scripts\activate`.

TIP No **.venv** folder (e.g. a new laptop)? Make one: `python3.12 -m venv .venv`, then `source .venv/bin/activate` and `pip install notebook pandas matplotlib python-docx pillow`.

0.2 Get the new files

- 1 Download everything I added (Labs 1-4 notebooks, scripts, worklogs):

```
TYPE THIS, then press Enter
git checkout claude/optimistic-cray-x20tfr
git pull
```

You should now have **Lab 1** > **Lab1_LLM_Access.ipynb**, **Lab 3** > **simple-rag** and **Lab 1** > **Lab1_Worklog_Week1.docx** etc. If git complains about **local changes**, close Word and run `git stash` first, then `git pull` again.

0.3 Install the libraries for Labs 1 and 3

- 1 With **(.venv)** on:

```
TYPE THIS, then press Enter
pip install ollama google-genai python-dotenv requests gradio pinecone
```

Takes a minute. It ends with **Successfully installed** (Lab 2's `smolagents` gets installed in Week 2 so you can screenshot it; Lab 4 gets its own environment.)

0.4 Make your .env file (where your keys go)

Each week you'll create a free key on a website. They all go in one file called **.env** in the LabChallenge folder. Git ignores this file, so it never ends up on GitHub.

- 1 Make it from the template and open it in TextEdit:

```
TYPE THIS, then press Enter
cp .env.example .env
open -e .env
```

- 2 You'll see lines like GEMINI_API_KEY=your_gemini_api_key. Leave it open – each week you replace one *your_...* part with your real key (no spaces, no quotes) and save with **Cmd** + **S**.

On Windows: copy `.env.example .env` then notepad `.env`.

TIP Files starting with a dot are hidden in Finder. To see them press **Cmd** + **Shift** + **.** in the Finder window.

KEEP YOUR KEYS SECRET Your keys go **only** in the **.env** file. Never type them into a notebook or a .py file, never show them in a screenshot, never send them to anyone. If one ever leaks, delete it on the website and make a new one.

0.5 Screenshots on a Mac

- 1 Press **Cmd** + **Shift** + **4** and drag a box around what the guide asks for (make sure the text is readable).
- 2 The picture lands on your **Desktop** as *Screenshot 2026-...png*. Drag it into **LabChallenge** > **Lab N** > **screenshots**, click its name once, press **Enter**, and type the name the guide gives (e.g. **1.1_ollama_cli**) – keep the **.png**.

TIP Only the number at the start has to be exact (**1.1_...**). Something too long for one picture? Take two and name them **1.2a_...** and **1.2b_...** – both get inserted.

On Windows: **Win** + **Shift** + **S**, click the pop-up, then save (Ctrl + S) straight into the screenshots folder.

0.6 Two Terminal tabs

Tab	Used for
Tab 1	Runs Jupyter (jupyter notebook). Once it's started leave it alone – don't close it, don't type in it.
Tab 2	Everything else. Open it with Cmd + T , then do 0.1 again (cd + activate).

Week 1 · LLM access – local + cloud

≈ 45 minutes · Lab 1 · dates in your worklog: 2-9 Aug

WHAT THIS WEEK IS ABOUT (in plain English)

An **LLM** (like ChatGPT) can run in two places: on **your own laptop** (free, private, but small) or in the **cloud** (big and fast, but you need an account and a key). You run the same questions on both – a small model with **Ollama** and Google's **Gemini** – and compare speed, cost and quality. The notebook does all the measuring for you.

1A Install Ollama and chat in Terminal

1 Go to **ollama.com/download** → **Download for macOS**. Open the download, drag **Ollama** into **Applications**, then open it from Applications (click **Open** if the Mac asks). A llama icon appears at the top of the screen – leave Ollama running.

2 In **Tab 2**:

```
TYPE THIS, then press Enter
ollama run llama3.2:1b
```

First time it downloads ~1.3 GB (*pulling ...* lines, then **success**). Then you get **>>>** – type **hello** and press **Enter**.

SCREENSHOT 1.1 save as 1.1_ollama_cli.png

The Terminal window: the *pulling ... success* lines and the model's reply to **hello**.

3 Type **/bye** and press **Enter** to leave the chat.

1B Make your Gemini key

1 Go to **aistudio.google.com/app/api-keys** and sign in with your Google account (accept the terms if asked).

2 Click **Create API key** → give it a name (e.g. *COS30018*) → choose the default project → **Create key**. Click the copy icon next to the key.

3 In your **.env** file (TextEdit), replace *your_gemini_api_key* with the key → **Cmd** + **S**.

OLD KEY Your old key was typed into your Lab 1 notebook, so treat it as leaked: on the API keys page click the bin/delete icon next to it, then make a new one (step 2) and use only the new one.

SCREENSHOT 1.3 save as 1.3_ai_studio_key.png

The API keys page with your key in the list – it only shows the last few characters, that's fine. **Not** the pop-up that shows the full key.

1C Your completed notebook

You've already done the Lab 1 notebook yourself. **Lab 1 > Lab1_completed.ipynb** is your notebook with only one change: the API key is taken out (it had been typed into the code). Your Week 1 worklog is already filled in from its outputs, so there's nothing to re-run.

- 1 In **Tab 1** (in LabChallenge, (.venv) on):

TYPE THIS, then press Enter
jupyter notebook

Your browser opens Jupyter. (If not, copy the **http://localhost:8888/...** line into the browser.)

- 2 Click **Lab 1 → Lab1_completed.ipynb**. **Don't** use Run All – the outputs are your results. (If you do re-run it, the times change and the worklog needs the new numbers.)
- 3 In the last cell (**Checkpoint Questions**) double-click, replace your table and answers with the corrected ones from your worklog ("Deliverable – comparison table" + "Checkpoint questions"), then press **Shift** + **Enter** and save with **Cmd** + **S**.
- 4 Take these screenshots from the notebook:

SCREENSHOT 1.2 save as 1.2_ollama_notebook.png

The **import ollama** + **ollama.chat(...)** cells and the start of the answer to 'What is an intelligent system?'.

SCREENSHOT 1.4 save as 1.4_gemini_notebook.png

The **genai.Client()** + **generate_content** cells and the start of Gemini's answer (no key visible in this copy).

SCREENSHOT 1.5 save as 1.5_count_both_models.png

Both counting functions' **Latency: 1.32s / Latency: 3.18s** lines and both answers. Two pictures OK: 1.5a_..., 1.5b_....

SCREENSHOT 1.6 save as 1.6_reasoning.png

The four apple cells with their Latency lines and answers. Two pictures OK: 1.6a_..., 1.6b_....

SCREENSHOT 1.8 save as 1.8_comparison_table.png

The Checkpoint Questions cell with the corrected comparison table.

- 5 The chat window: click the **import ollama** cell near the top and press **Shift** + **Enter**, then do the same for the three Gradio cells at the bottom. Open **http://127.0.0.1:7860** and ask it something.

SCREENSHOT 1.7 save as 1.7_gradio_chat.png

The 'Week 1: Local LLM Chat (Ollama)' page with an answer.

Finish your Week 1 worklog

Do these **in this order** – the first step fills the worklog with your real numbers, and it only works while the worklog is still untouched.

1 **Close Word** if the worklog is open. In **Tab 2** make sure you're in the **LabChallenge** folder with **(.venv)** at the start of the line (if not: `cd ~/Documents/LabChallenge` then `source .venv/bin/activate`).

2 Fill the worklog with the numbers from **your** run:

```
TYPE THIS, then press Enter  
python _tools/build_worklogs.py 1
```

It should say **week 1: wrote Lab 1/Lab1_Worklog_Week1.docx**. The yellow placeholders are now your real results.

3 Put your screenshots in:

```
TYPE THIS, then press Enter  
python _tools/insert_screenshots.py 1
```

It lists what it **inserted** and what's **still missing**. Missing? Check the file is in **Lab 1 > screenshots** and its name starts with the exact number.

4 Tick the checklist items only you can know about:

```
TYPE THIS, then press Enter  
python _tools/tick_checklist.py 1 10
```

(Run `python _tools/tick_checklist.py 1` on its own to see all items with their numbers.)

5 Open **Lab 1 > Lab1_Worklog_Week1.docx** (double-click in Finder). Read it once – it's written as you, so change anything you wouldn't say. Change the **TIME SPENT** numbers to how long it really took you. Save with `Cmd + S`.

6 Make the clean copy for your tutor (no checklist, no notes to yourself):

```
TYPE THIS, then press Enter  
python _tools/make_handin.py 1
```

It writes **Completed Labs > Lab 1 Completed.docx** and warns you if a screenshot or a yellow placeholder is still missing.

Week 2 · An agent with tools and memory

≈ 1 hour · Lab 2 · dates in your worklog: 10–16 Aug

WHAT THIS WEEK IS ABOUT (in plain English)

Instead of just asking a model a question, an **agent** can use **tools** (here: web search) and works in a loop: it **thinks**, writes some **code**, looks at the **result**, and repeats until it has the answer. The **smolagents** library runs that loop for you. You run it with a big cloud model (via Hugging Face) and a small model on your laptop, and compare.

2A Hugging Face token

- 1 Make a free account at **huggingface.co** (confirm your email).
- 2 Click your profile picture (top right) → **Settings** → **Access Tokens** → + **Create new token** → choose **Read** → name it (e.g. *COS30018*) → **Create token** → **Copy** (it's only shown once!).
- 3 In **.env** replace *hf_your_token* with it → save.

SCREENSHOT 2.1 save as 2.1_hf_token.png

The Access Tokens page with your token in the list (value hidden). Not the pop-up that shows the full token.

2B Install smolagents

- 1 In **Tab 2** (|.venv) on):

TYPE THIS, then press Enter

```
pip install "smolagents[toolkit]" "smolagents[transformers]"
```

This one is big (it brings PyTorch + transformers if you don't have them) – a few minutes.

SCREENSHOT 2.2 save as 2.2_pip_install.png

The end of the install (**Successfully installed ...**).

TIP If Jupyter was already running, restart it after installing: in Tab 1 press **Ctrl + C**, then **jupyter notebook** again.

2C Run the Lab 2 notebook

- 1 In Jupyter open **Lab 2** → **Lab2_smolagents_agent.ipynb** → **Kernel** › **Restart Kernel and Run All Cells...**

- 2** Wait. The cloud parts take seconds; **Step 5 (local model) downloads ~3 GB the first time and then runs on your CPU** – it can take 10+ minutes. If it fails, that's OK (the sheet says it might) – the error is saved and goes into your worklog.

TIP Laptop with only 8 GB of memory, or no time? Change `RUN_LOCAL = True` to `RUN_LOCAL = False` in the first code cell and run all again – the worklog will say you skipped it.

- 3** When the last cell prints **saved results/lab2_results.json**, take the screenshots:

SCREENSHOT 2.3 save as `2.3_no_tools.png`

Step 2: the **New run** box, Step 1 / Step 2 with the code, **Final answer: 1275**, and the small table.

SCREENSHOT 2.4 save as `2.4_web_search.png`

Step 3: Step 1 with **web_search(...)** + its Execution logs, and Step 2 with the 15% maths + Final answer. Two pictures OK: 2.4a_..., 2.4b_....

SCREENSHOT 2.5 save as `2.5_memory.png`

Step 4 output (TaskStep / ActionStep lines + 'what smolagents records') and the follow-up's last line (*searched again: False ...*).

SCREENSHOT 2.6 save as `2.6_local_model.png`

Step 5: **loaded Qwen/... in ... s**, the steps and the answer (or the error).

SCREENSHOT 2.7 save as `2.7_cloud_vs_local.png`

Step 6: the **Cloud: ...s / Local: ...s** lines and the comparison table.

- 4** Open the link printed under Step 7 (usually **`http://127.0.0.1:7860`**, or 7861 if the Week 1 chat is still running). Ask something that needs a search, e.g. *What's the population of Sydney? Then divide it by 4.*

SCREENSHOT 2.8 save as `2.8_gradio_ui.png`

The GradioUI chat with the agent's steps and the final answer.

WATCH OUT Cloud model error like **402, credits** or **quota**? Hugging Face's free monthly credit ran out – wait until next month or try again tomorrow; everything else still works.

Finish your Week 2 worklog

Do these **in this order** – the first step fills the worklog with your real numbers, and it only works while the worklog is still untouched.

- 1** **Close Word** if the worklog is open. In **Tab 2** make sure you're in the **LabChallenge** folder with **(.venv)** at the start of the line (if not: `cd ~/Documents/LabChallenge` then `source .venv/bin/activate`).

2 Fill the worklog with the numbers from **your** run:

TYPE THIS, then press Enter

```
python _tools/build_worklogs.py 2
```

It should say **week 2: wrote Lab 2/Lab2_Worklog_Week2.docx**. The yellow placeholders are now your real results.

3 Put your screenshots in:

TYPE THIS, then press Enter

```
python _tools/insert_screenshots.py 2
```

It lists what it **inserted** and what's **still missing**. Missing? Check the file is in **Lab 2 > screenshots** and its name starts with the exact number.

4 Tick the checklist items only you can know about:

TYPE THIS, then press Enter

```
python _tools/tick_checklist.py 2 11
```

(Run `python _tools/tick_checklist.py 2` on its own to see all items with their numbers.)

5 Open **Lab 2 > Lab2_Worklog_Week2.docx** (double-click in Finder). Read it once – it's written as you, so change anything you wouldn't say. Change the **TIME SPENT** numbers to how long it really took you. Save with `Cmd + S`.**6** Make the clean copy for your tutor (no checklist, no notes to yourself):

TYPE THIS, then press Enter

```
python _tools/make_handin.py 2
```

It writes **Completed Labs > Lab 2 Completed.docx** and warns you if a screenshot or a yellow placeholder is still missing.

Week 3 · RAG with a vector database

≈ 1.5 hours · Lab 3 · dates in your worklog: 17-23 Aug

WHAT THIS WEEK IS ABOUT (in plain English)

RAG = before answering, look up the right part of *your own documents* and give it to the model. Step 1 (**ingestion**): cut the text into chunks, turn each chunk into numbers (an **embedding**) and store them in **Pinecone**. Step 2 (**query**): turn the question into numbers, find the most similar chunks, and let Gemini answer only from them. You build it twice: first by clicking in **Make.com**, then in **Python**.

3A Pinecone (the vector database)

- 1 Sign up for free at **pinecone.io** (the free *Starter* plan).
- 2 Click **Create index**. Name: **intelligent-systems**. Choose the custom / manual settings: **Vector type: Dense, Dimension: 1024, Metric: cosine**. Capacity: **Serverless, AWS, us-east-1**. Click **Create index**.

WATCH OUT The dimension **must be 1024** – it has to match the embeddings in both the Make blueprints and the Python code.

SCREENSHOT 3.1 save as 3.1_pinecone_index.png

Your index page showing dimension 1024, metric cosine, dense.

- 3 Left menu → **API Keys** → **Create API key** → copy it. In **.env**: **PINECONE_API_KEY**=the key, and **PINECONE_INDEX**=intelligent-systems → save.

TIP Python way to make the index instead: `cd "Lab 3/simple-rag"` then `python setup_index.py` (after the key is in **.env**). You still need screenshot 3.1.

3B Make.com - Part 1: ingestion

- 1 Upload the text file to your Google Drive: **drive.google.com** → + **New** → **File upload** → pick **LabChallenge** > **Lab 3** > **intelligent_agents_overview.txt**.
- 2 Sign up for free at **make.com**. Go to **Scenarios** → + **Create a new scenario**.
- 3 Click the **...** (three dots) button → **Import blueprint** → choose **Lab 3** > **RAG Ingestion.blueprint.json** → **Save**. Five modules appear.

- 4** The blueprint still has the tutor's accounts, so click each module and connect yours:
- **Google Drive** → Connection: **Add** → sign in with Google → allow. Then for **File** pick your uploaded *intelligent_agents_overview.txt*.
 - **Google Gemini AI** → Connection: **Add** → paste your Gemini key (same as .env). Check **Output dimensionality = 1024**.
 - **Pinecone** → Connection: **Add** → paste your Pinecone key; choose your index if it asks. Check namespace **week3-lab**.
- Click **OK** on each, then save the scenario (`Cmd` + `S`).

KEEP YOUR KEYS SECRET Your keys go **only** in the **.env** file. Never type them into a notebook or a .py file, never show them in a screenshot, never send them to anyone. If one ever leaks, delete it on the website and make a new one.

- 5** Click **Run once** (bottom left). Each module gets a green tick and a bubble with a number.

SCREENSHOT 3.2 save as 3.2_make_ingestion.png

The scenario after Run once – all 5 modules with ticks and bubbles.

- 6** Pinecone console → your index → **Browser** tab → Namespace **week3-lab** → click a record so you can see its **text** and **source**.

SCREENSHOT 3.3 save as 3.3_make_pinecone_record.png

(deliverable) A record from namespace **week3-lab** with its ID and **text** + **source** metadata.

3C Make.com – Part 2: query

- 1** New scenario → ... → **Import blueprint** → **Lab 3** > **RAG Query.blueprint.json**. Connect your accounts again (both Gemini modules + Pinecone), like in 3B.
- 2** Click the first module (**Set variable**) → change the value to **What is a learning agent?** → OK → **Run once**. Click the bubble on the **last** module to see the answer.

SCREENSHOT 3.4 save as 3.4_make_query_answer.png

(deliverable) The query scenario after Run once, with the answer bubble open (like the last picture in the Make sheet).

- 3** Change the question to **What's the weather in Melbourne?** → Run once → open the answer.

SCREENSHOT 3.5 save as 3.5_make_out_of_scope.png

The answer saying it doesn't have enough information.

- 4** The modification: question back to **What is a learning agent?**, click the **Pinecone** module → change **Limit** from 3 to **1** → OK → Run once. Then try **5**. Put it back to 3 afterwards.

SCREENSHOT 3.6 save as 3.6_make_limit_change.png

The Pinecone module settings with the new Limit + the answer bubble after running. Two pictures OK: 3.6a_..., 3.6b_....

3D Python - the same thing in code

- 1** In **Tab 2** (.venv) on, in LabChallenge):

```
TYPE THIS, then press Enter
cd "Lab 3/simple-rag"
python concept_demo.py
```

Asks Gemini a question about the notes **without** and **with** the right chunk pasted in.

SCREENSHOT 3.7 save as 3.7_concept_demo.png

The output: the answer with NO context and WITH the chunk.

- 2** Part 1 – ingestion:

```
TYPE THIS, then press Enter
python ingest.py
```

Ends with **Ingestion complete.** and **Pinecone now has 9 records**

SCREENSHOT 3.8 save as 3.8_ingest.png

The ingest.py output (Loaded..., Created 9 chunks, Processing chunk 1/9 ... 9/9, Ingestion complete).

- 3** Pinecone console → Browser → namespace **week3-code-lab** → click a record.

SCREENSHOT 3.9 save as 3.9_python_pinecone_record.png

(deliverable) A record with **text**, **source** and **chunk_number**.

- 4** Part 2 – query:

```
TYPE THIS, then press Enter
python query.py
```

At **Ask a question:** type What is a learning agent? and press **Enter**.

SCREENSHOT 3.10 save as 3.10_query_learning_agent.png

(deliverable) The retrieved chunks (Rank, ID, Score, Text) and the FINAL ANSWER. Two pictures OK: 3.10a_..., 3.10b_....

- 5** The out-of-scope question:

```
TYPE THIS, then press Enter
python query.py "What's the weather in Melbourne?"
```

SCREENSHOT 3.11 save as 3.11_query_out_of_scope.png

(deliverable) The low scores and the 'not enough information' answer.

- 6** The experiments (top-k 1/3/5, chunk size 100/400, more questions) – a few minutes:

```
TYPE THIS, then press Enter
python experiments.py
```

SCREENSHOT 3.12 save as 3.12_experiments.png

(deliverable) The Experiment A part and the Experiment B part of the output. Two pictures OK: 3.12a_..., 3.12b_....

- 7 Check what Make stored and repeat its search with Limit 1/3/5 (this fills the Make part of your worklog):

```
TYPE THIS, then press Enter
python make_check.py
cd ../..
```

The last line takes you back to the LabChallenge folder.

WATCH OUT Model not found / 404? Google renamed a model – the scripts try the next one by themselves. **429 / rate limit?** They wait and retry; if it keeps happening, wait a few minutes and run that script again.

Finish your Week 3 worklog

Do these **in this order** – the first step fills the worklog with your real numbers, and it only works while the worklog is still untouched.

- 1 **Close Word** if the worklog is open. In **Tab 2** make sure you're in the **LabChallenge** folder with **(.venv)** at the start of the line (if not: `cd ~/Documents/LabChallenge` then `source .venv/bin/activate`).

- 2 Fill the worklog with the numbers from **your** run:

```
TYPE THIS, then press Enter
python _tools/build_worklogs.py 3
```

It should say **week 3: wrote Lab 3/Lab3_Worklog_Week3.docx**. The yellow placeholders are now your real results.

- 3 Put your screenshots in:

```
TYPE THIS, then press Enter
python _tools/insert_screenshots.py 3
```

It lists what it **inserted** and what's **still missing**. Missing? Check the file is in **Lab 3 > screenshots** and its name starts with the exact number.

- 4 Tick the checklist items only you can know about:

```
TYPE THIS, then press Enter
python _tools/tick_checklist.py 3 7 8 13
```

(Run `python _tools/tick_checklist.py 3` on its own to see all items with their numbers.) Items 7 and 8 are the Make query runs you did in 3C.

- 5 Open **Lab 3 > Lab3_Worklog_Week3.docx** (double-click in Finder). Read it once – it's written as you, so change anything you wouldn't say. Change the **TIME SPENT** numbers to how long it really took you. Save with `Cmd + S`.

6

Make the clean copy for your tutor (no checklist, no notes to yourself):

TYPE THIS, then press Enter

```
python _tools/make_handin.py 3
```

It writes **Completed Labs > Lab 3 Completed.docx** and warns you if a screenshot or a yellow placeholder is still missing.

Week 4 · Multi-agent conversation (AutoGen)

≈ 45 minutes · Lab 4 · dates in your worklog: 24-30 Aug

WHAT THIS WEEK IS ABOUT (in plain English)

Two AI **agents** talk to each other: the **assistant** writes Python code, the **userproxy** runs it on your laptop and reports back, until the job is done. It's the tutor's AutoGen app with a chat page. I fixed the bits that stopped it working with Gemini (they're listed in your worklog) and added a script that runs the examples for you, plus an extra agent (a **critic**).

WHY A SECOND ENVIRONMENT This lab needs **its own environment with Python 3.12**: the tutor's pyautogen==0.2.28 doesn't install on Python 3.13+, and its old Gradio version would break Labs 1-2 if you put it in .venv.

4A Make the AutoGen environment

- 1 In **Tab 2**, switch .venv off and make the new one (in the LabChallenge folder):

```
TYPE THIS, then press Enter
deactivate
python3.12 -m venv .venv-autogen
source .venv-autogen/bin/activate
```

The line now starts with **(.venv-autogen)**. python3.12: command not found? Install Python 3.12 from python.org/downloads/macos (Python 3.12.x, macOS installer) and try again.

- 2 Install the tutor's requirements + yfinance (for the stock example):

```
TYPE THIS, then press Enter
cd "Lab 4/Multiagent"
pip install -r requirements.txt yfinance
python --version
```

SCREENSHOT 4.1 save as 4.1_venv_install.png

Successfully installed ... and Python 3.12.x from the last command.

On Windows: py -3.12 -m venv .venv-autogen and .venv-autogen\Scripts\activate.

4B Show the fix

- 1 Open **Lab 4 > Multiagent > CONFIG_LIST.json** and **agent.py** (in VS Code, or right-click → Open With → TextEdit). Your key is **not** in them – it comes from .env.

SCREENSHOT 4.2 save as 4.2_config_fix.png

CONFIG_LIST.json with the **base_url** line, and the **(my fix)** lines in agent.py. Two pictures OK: 4.2a_..., 4.2b_....

4C Run the app

- 1 Still in **Lab 4/Multiagent** with `(.venv-autogen):`

```
TYPE THIS, then press Enter
python app.py
```

Wait for **Running on local URL: http://0.0.0.0:7868**.

SCREENSHOT 4.3 save as `4.3_app_running.png`

The Terminal with the **Running on local URL** line.

- 2 Open **http://127.0.0.1:7868** in your browser. Click the examples **one at a time, in order** and wait for each answer (10-60 s).

SCREENSHOT 4.4 save as `4.4_example_sum.png`

Example 1: the assistant's code, the userproxy's **exitcode: 0**, the final answer.

SCREENSHOT 4.5 save as `4.5_example_product.png`

Example 2 (*what if the production...*): it should use multiplication.

SCREENSHOT 4.6 save as `4.6_example_stock_chart.png`

Example 3 (stock chart) and example 4 (**show file:** with the chart in the chat). Two pictures OK: 4.6a_..., 4.6b_....

- 3 Back in Terminal press `Ctrl + C` to stop the app.

4D The scripts that fill your worklog

- 1 Run the 4 examples without the browser (saves the numbers + full conversations):

```
TYPE THIS, then press Enter
python run_examples.py
```

SCREENSHOT 4.7 save as `4.7_run_examples.png`

The end of the output: the `--> ... s | ... messages` line for each example and **saved ...**

- 2 The extra agent - coder + critic + executor:

```
TYPE THIS, then press Enter
python groupchat_demo.py
```

SCREENSHOT 4.8 save as `4.8_groupchat.png`

Next speaker: critic, the critic's **APPROVED** (or its comments), the exit code and the summary line.

- Go back to the main environment for the worklog tools:

```
TYPE THIS, then press Enter
deactivate
cd ../..
source .venv/bin/activate
```

WATCH OUT Timeout Error or an error about the key → check GEMINI_API_KEY in .env, then restart app.py. **No matching distribution for pyautogen** → you're not on Python 3.12 (redo 4A). The stock example says yfinance is missing → `pip install yfinance` in (.venv-autogen).

Finish your Week 4 worklog

Do these **in this order** – the first step fills the worklog with your real numbers, and it only works while the worklog is still untouched.

- Close Word** if the worklog is open. In **Tab 2** make sure you're in the **LabChallenge** folder with **(.venv)** at the start of the line (if not: `cd ~/Documents/LabChallenge` then `source .venv/bin/activate`).

- Fill the worklog with the numbers from **your** run:

```
TYPE THIS, then press Enter
python _tools/build_worklogs.py 4
```

It should say **week 4: wrote Lab 4/Lab4_Worklog_Week4.docx**. The yellow placeholders are now your real results.

- Put your screenshots in:

```
TYPE THIS, then press Enter
python _tools/insert_screenshots.py 4
```

It lists what it **inserted** and what's **still missing**. Missing? Check the file is in **Lab 4 > screenshots** and its name starts with the exact number.

- Tick the checklist items only you can know about:

```
TYPE THIS, then press Enter
python _tools/tick_checklist.py 4 5 9
```

(Run `python _tools/tick_checklist.py 4` on its own to see all items with their numbers.) Item 5 is trying the examples in the browser (4C).

- Open **Lab 4 > Lab4_Worklog_Week4.docx** (double-click in Finder). Read it once – it's written as you, so change anything you wouldn't say. Change the **TIME SPENT** numbers to how long it really took you. Save with `Cmd + S`.

- Make the clean copy for your tutor (no checklist, no notes to yourself):

```
TYPE THIS, then press Enter
python _tools/make_handin.py 4
```

It writes **Completed Labs > Lab 4 Completed.docx** and warns you if a screenshot or a yellow placeholder is still missing.

Finish - hand in and save

≈ 15 minutes

- 1 Open each **Completed Labs > Lab N Completed.docx** (N = 1-4) once and check: your name + ID at the top, the dates, no yellow boxes or yellow text left, the screenshots are readable.
- 2 If your tutor wants PDFs: in Word **File > Save As...** → File Format **PDF**.
- 3 Save everything to GitHub (your results, screenshots and worklogs - **.env is ignored**, so your keys stay on your laptop):

TYPE THIS, then press Enter

```
git add -A
git commit -m "Labs 1-4 results, screenshots and worklogs"
git push
```

Before pushing, `git status` must **not** list a file called **.env**. (It won't - it's in .gitignore.)

- 4 Hand in what your tutor asked for (the worklogs, plus for Week 3 the deliverable screenshots and notes - they're already inside your Week 3 worklog).

TIP A lot of this was prepared with an AI assistant. Read each worklog's answers so you can explain them yourself, and check your unit's rules on AI use - add an acknowledgement if they ask for one.

Troubleshooting

What you see	What to do
command not found: python	Use python3 - or activate the environment first (the line must start with (.venv)).
ModuleNotFoundError in a notebook	Jupyter was started without (.venv), or that week's install was skipped. In Tab 1: Ctrl + C , activate, install, jupyter notebook.
KeyError / 'No HF_TOKEN' / 'Missing GEMINI_API_KEY...'	The key isn't in .env, .env isn't saved, or it's not in the LabChallenge folder. No spaces or quotes around the key.
429 / RESOURCE_EXHAUSTED / rate limit	Free-tier limit. The code waits and retries. If it says per day , the notebook switches to another Gemini model by itself; otherwise try tomorrow.
404 ... model not found	Google renamed a model. The notebooks/scripts move to the next one in their list automatically.
build_worklogs says has been edited ... NOT overwriting	You opened/changed the worklog (or inserted screenshots) before building it. Easiest fix: <code>git checkout -- "Lab N/LabN_Worklog_WeekN.docx"</code> (puts the original back), then build → insert screenshots → tick, in that order.
insert_screenshots says still missing	File not in Lab N > screenshots , or the name doesn't start with the exact number (e.g. 3.10_), or it isn't .png/.jpg.
PermissionError from a _tools command	The worklog is open in Word - close it and run again.

What you see	What to do
Make.com: a module shows a red error	Usually a connection: click the module → Connection → Add → paste your key again. For Google Drive, re-pick the file.
Pinecone: dimension ... does not match	The index isn't 1024. Make a new index with dimension 1024 and put its name in .env.
Port 7860 / 7868 already in use	An old chat is still running. Close the old Jupyter/Terminal or use the other link Gradio prints.